A.J. INSTITUTE OF ENGINEERING & TECHNOLOGY, MANGALURU

Department of MCA

SOFTWARE DESIGN SPECIFICATION (SDS)

LEGALAI – AI-BASED LEGAL ASSISTANT WITH RETRIEVAL-AUGMENTED GENERATION (RAG)

Name: Lekhan N U

USN: 4JK25MC053

Guide: Mrs Ramya Prabhu

1. System Design

1.1 Architectural Design

LegalAI is made of separate parts that talk to each other through clear interfaces. There is a web front end built with HTML, CSS and JavaScript, a Flask backend, a retrieval component that searches a FAISS vector index, the Google Gemini API for writing answers, and a MySQL database for users, chats and document records. The browser talks to the backend over HTTPS. The backend holds the rules: who can do what, how a question is checked, how many passages are retrieved and when the system should say that no source was found. Answers are produced with Retrieval-Augmented Generation (RAG), where the most relevant legal passages are found first and then given to the language model as context [1], [2], [5]. Keeping retrieval, generation and storage in separate components means the embedding model, the language model or the index can be changed without rewriting the website.

1.1.1 System Context Diagram

The context diagram shows who uses the system and what it connects to. General users and administrators only ever deal with the web application. Behind it, the backend talks to the Gemini API for generating answers, to the FAISS index for similarity search and to MySQL for records. Legal documents are collected offline from reliable sources and loaded by the administrator [3], [4].

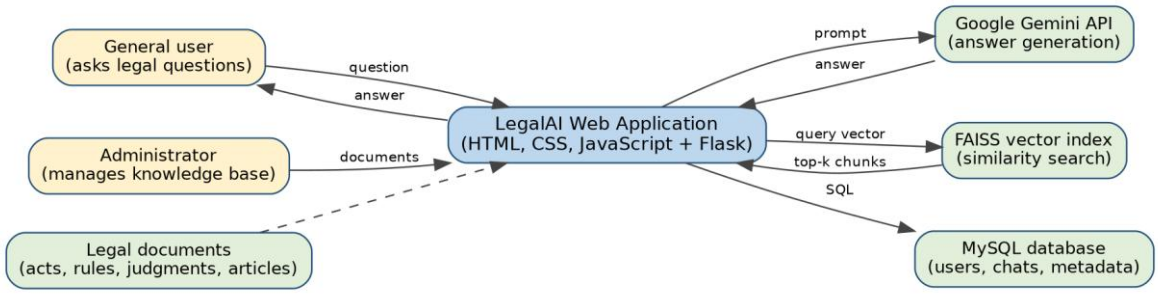


Figure 1.1.1: System context diagram

External actor / system	Interaction with platform	Protocol / format
General user	Registers, logs in, asks legal questions, views answers and sources, gives feedback	HTTPS, JSON, JWT
Administrator	Uploads legal documents, rebuilds the index, manages users and views logs	HTTPS, JSON, multipart upload, JWT
Google Gemini API	Generates an easy-to-read answer from the question and retrieved context	HTTPS REST API
FAISS vector index	Stores chunk embeddings and returns the nearest chunks for a query	In-process library, index file on disk
MySQL database	Stores users, chat history, document and chunk metadata, feedback and audit logs	SQL over TCP, encrypted connection
Legal sources	Provide acts, rules, regulations, judgments and legal articles as PDF or text	Offline download, licence-checked

1.1.2 End-to-End RAG Pipeline

The system runs as two repeatable paths. In the indexing path, documents are collected, text is extracted with PyPDF, cleaned, split into overlapping chunks, converted into embeddings with Sentence Transformers [6] and stored in FAISS [7], while the chunk text and metadata go to MySQL. In the query path, the question is checked and embedded, the closest chunks are retrieved, a prompt is built with LangChain and sent to Gemini, and the answer is shown together with its sources. Every indexing run records the document hash and the settings used, so any answer can be traced back to the exact passages it came from.

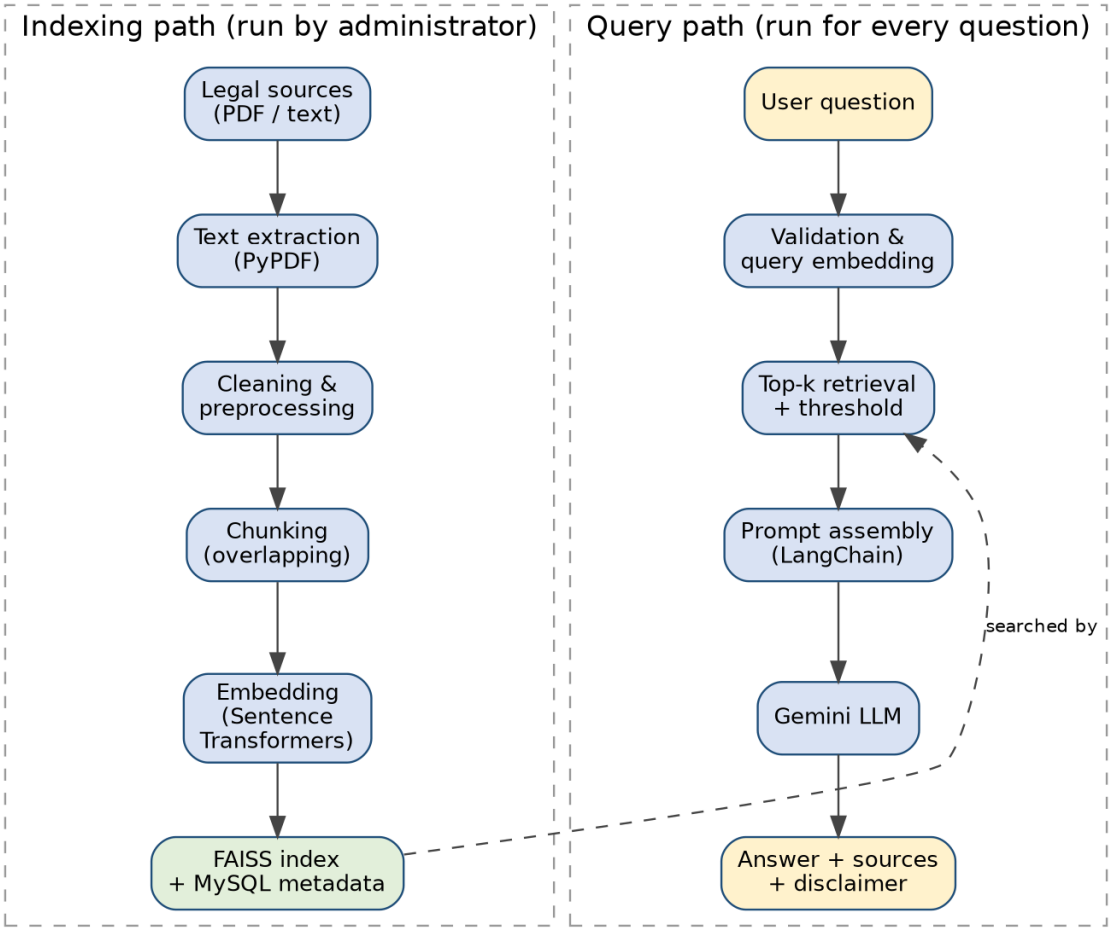


Figure 1.1.2: End-to-end RAG pipeline

2. Detailed Design

2.1 Knowledge Base Design

LegalAI does not need a conventional numerical machine-learning training dataset. It uses a curated legal knowledge base that is searched at question time. This section describes where the legal text comes from, how it is described, what the system is expected to return and how the evaluation data is kept separate from the data used for tuning.

2.1.1 Source Provenance and Permissions

Before a document is indexed, we record where it came from, who published it, what permission covers it, when it was collected and what it will be used for. The table below lists these details for each source type.

Source	Owner / publisher	Licence / permission	Collection period	Intended use
Acts and laws	Government of India and State legislatures (official publications)	Public legal text; reuse terms to be verified on the official site	Data collection phase of the project plan	Retrieval and answering; educational use
Rules and regulations	Ministries and regulatory bodies	Public notifications; verify terms before use	Data collection phase	Retrieval and answering
Judgments	Courts (official judgment portals)	Public records; verify terms of the portal	Data collection phase	Retrieval; case title, court and date shown as source
Legal articles	Publishers and legal organisations	Only openly licensed or permitted articles; URL recorded	Data collection phase	Explanations and context
Documents added by administrator	Uploaded by the administrator	Administrator confirms the right to use; terms recorded in the datasets record	Continuous after deployment	Extending the knowledge base

Licence terms change over time, so each entry should be checked on the source's official page before the final report goes in. User questions are kept separate from the knowledge base and are used only to show chat history and to review answer quality. The system gives general legal information and does not replace professional legal advice.

2.1.2 Knowledge Base Schema and Data Dictionary

Every field in a document and chunk record is described below. The same IDs appear in the MySQL tables and the FAISS position map, so a retrieved vector can always be matched to its text and source. Sensitivity shows how much harm a leak could cause: Low, Medium or High.

Field	Type	Meaning	Permissible range	Missingness	Sensitivity
doc_id	Integer	Unique identifier of a document	Unique, positive	Not allowed	Low
title	String	Act, rule, judgment or article title	1 to 300 characters	Not allowed	Low
category	Category	Type of document	act, rule, judgment, article, other	Not allowed	Low
source_name	String	Publisher or portal name with URL	Non-empty	Not allowed	Low
court, judgment_date	String, Date	Court and date for judgments	Valid date	Allowed except for judgments	Low
sha256	String	Hash of the file for duplicate detection	64 hexadecimal characters	Not allowed	Low
status	Category	Processing state of the document	pending, indexed, rejected	Not allowed	Low
chunk_id	Integer	Unique identifier of a chunk	Unique, positive	Not allowed	Low

Field	Type	Meaning	Permissible range	Missingness	Sensitivity
chunk_text	Text	Cleaned text of the chunk	50 to 2,000 characters	Not allowed	Low
page_no, section_ref	Integer, String	Page and section/provision where the chunk appears	Page ≥ 1	Allowed (shown as unavailable)	Low
faiss_pos	Integer	Position of the chunk's vector in the FAISS index	0 to index size − 1	Not allowed	Low
embedding	Float32 vector	384-value vector of the chunk	Fixed length 384	Not allowed	Low
question_text	Text	Question typed by the user	3 to 1,000 characters	Not allowed	High (may contain personal details)

2.1.3 Target / Label Definition

There is no supervised label to learn. The system returns three things: the most relevant chunks, a grounded answer, and an uncertain flag when nothing relevant is found. To check that these are correct, an evaluation set of questions is prepared. Each question is paired with the document section that should answer it and a short reference answer. These pairs are written from the source text and checked by the project guide, not produced by the model. If the model's own answers were used as references, the evaluation would only measure agreement with itself.

- Each question is linked to the expected act, section or judgment, so retrieval can be scored automatically.
- Retrieval is measured with Recall@5 and mean reciprocal rank. The target is Recall@5 of 0.80 or higher.
- Answers are reviewed by hand for faithfulness, meaning every statement is supported by the retrieved text. The target is 90% or higher.
- An answer is marked uncertain when the best retrieval score is below the threshold; in that case the system says that no relevant source was found instead of guessing.

2.1.4 Partitioning and Leakage Prevention

The whole knowledge base is indexed, so the split applies to the evaluation questions. A careless split can leak information when several questions point to the same section, so each dataset gets the split that suits it.

Dataset	Split	Proportion	Strategy and reason
Knowledge base chunks	No split; all documents indexed	100%	Duplicates removed first so the same passage is not stored twice
Evaluation questions	Tuning / final test	40% / 60%	Grouped by source document so questions about the same section never appear in both sets

Dataset	Split	Proportion	Strategy and reason
Feedback and chat logs (future review)	Older / newer	70% / 30%	Time-based: oldest logs for tuning and newest for checking, which mirrors real use

These checks keep the reported results honest:

- Exact duplicates are found with SHA-256 hashes and near-identical chunks with a text similarity check. Only one copy is kept, and this happens before evaluation.
- The threshold, chunk size and top-k value are tuned on the tuning questions only.
- The reference answer or expected section is never placed in the prompt.
- The final test set is used once per configuration, after tuning is finished.

2.2 Preprocessing and Feature Engineering

2.2.1 Validation and Cleaning

The same checks run on administrator uploads and on every live question. Documents that fail are set aside and logged. A live question that fails a blocking check is sent back with a clear message. Problems that are not blocking are flagged, and the request still goes through.

- Only PDF and plain-text files are accepted. Files that cannot be opened or contain no extractable text are rejected and logged as corrupt.
- Scanned PDFs with no text layer are flagged as needing OCR and are not indexed until text is available.
- Headers, footers, page numbers, repeated line breaks and extra spaces are removed. Section numbers and headings are kept, because they are used as source references.
- Text is normalised to UTF-8; legal words such as “not”, “shall” and “except” are never removed, because they change the meaning of a provision.
- Chunks shorter than 50 characters are merged with a neighbour. Chunks longer than 2,000 characters are split again.
- An empty question or one longer than 1,000 characters is rejected. Questions in unsupported languages are answered with a notice that only English is currently supported.

2.2.2 Feature Selection / Extraction

Each feature was picked for three reasons: it helps retrieval, it can be explained to users, and it is quick enough for a normal laptop.

Feature	Source	Extraction method	Rationale
Chunk text	Document	Recursive splitter, about 800 characters with 100 overlap	Keeps a provision together while the overlap avoids cutting meaning at the edges
Chunk embedding (384-d)	Chunk text	Sentence Transformers model [6]	Captures meaning, so a question can match text with different wording

Feature	Source	Extraction method	Rationale
Query embedding (384-d)	User question	Same model as the chunks	Both must use one model so distances are comparable
Similarity score	Query and chunk	Cosine similarity through FAISS inner product on normalised vectors [7]	Ranks chunks and drives the relevance threshold
Source metadata	MySQL	Document title, category, page and section	Shown to the user so every answer can be checked
Question length and language check	User question	Character count and simple language detection	Blocks empty or unsupported input before any API call is made

2.3 Model Design

2.3.1 Baseline and Candidate Models

Every task starts with a simple baseline. A more complex option is only adopted if it clearly beats that baseline. Because the corpus is small, pretrained models are used as they are and no model is trained from scratch [5], [6].

Task	Baseline	Candidate models	Selected model	Justification
Text embedding	TF-IDF with cosine similarity	all-MiniLM-L6-v2; all-mpnet-base-v2; multilingual MiniLM	all-MiniLM-L6-v2 [6]	Small and fast on a Core i3 with 8 GB RAM; multilingual models are kept for the multi-language future scope
Vector search	Brute-force cosine search with NumPy	FAISS IndexFlatIP; FAISS IVF index	FAISS IndexFlatIP [7]	Exact results and no tuning needed for a corpus of this size; IVF can be added if it grows
Chunking	Fixed 500-character windows	Recursive splitter with overlap	Recursive splitter, 800 / 100	Keeps sentences and sections together, which improves retrieval
Answer generation	Show retrieved passages only	Gemini without retrieval; Gemini with RAG	Gemini with RAG through LangChain	Grounded answers with sources; an LLM without retrieval can give unsupported legal responses [3]

2.3.2 Model Architecture

The system has one embedding model, one retriever and one language model. The embedding model turns the question into 384 values. The retriever returns the five closest chunks and passes them on only if the best score reaches the threshold. A prompt template then tells Gemini to answer only from the given context, to keep the answer simple and to refuse when the context is not enough. Everything comes out as one JSON result with the answer, the sources, an uncertain flag and a disclaimer, so the screen can label the output as general information. The block diagram shows the layout and the table gives the details of each part.

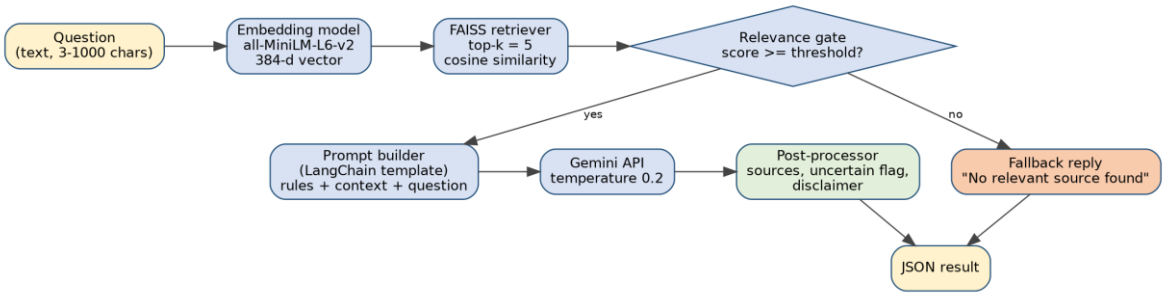


Figure 2.3.2: Block diagram

Component	Input	Algorithm / setting	Output	Approx. size
Embedding model	Question or chunk text	all-MiniLM-L6-v2, mean pooling, normalised	384-d vector	≈ 22 M parameters
FAISS retriever	Query vector	IndexFlatIP, top-k = 5	Chunk positions and scores	384 × 4 bytes per chunk
Relevance gate	Best score	Threshold 0.35 (initial value)	Continue or fallback	0
Prompt builder	Question, chunks, metadata	LangChain template with answer-only-from-context rules	Prompt text	0
Language model	Prompt	Google Gemini API, temperature 0.2, up to 600 output tokens	Answer text	Hosted service
Post-processor	Answer, chunks	Attach sources, set uncertain flag, add disclaimer	JSON with answer, sources, uncertain, modelName	0

Initial settings: chunk size 800 characters with 100 overlap, top-k of 5, relevance threshold 0.35, temperature 0.2 and a request timeout of 30 seconds with two retries. These are starting values and are tuned on the tuning questions only.

2.4 Database Design

2.4.1 ER Diagram / Storage Architecture

MySQL was chosen because the data is structured and closely related: a user has many chats, a chat has many questions, and every answer points to the chunks it used. Foreign keys and transactions keep these links correct. Vectors are not kept in MySQL. They are stored in the FAISS index file, and each chunk row keeps its position in that index. Uploaded files are kept in a private folder on the server, and the database stores only their path and hash. Collections refer to each other by ID instead of copying data, so a correction has to be made in one place only.

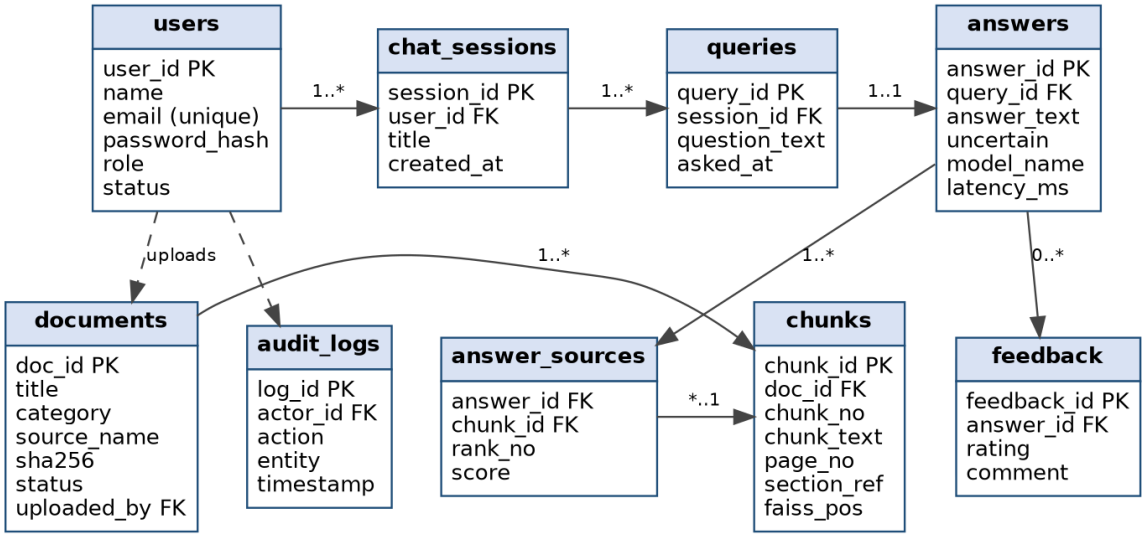


Figure 2.4.1: Entity-relationship diagram

The table below shows where each kind of data is kept and why.

Type of data	Storage location	Reason
Users, chats, questions, answers, feedback, audit logs	MySQL (password hashes, restricted database user, TLS connection)	Related records with integrity rules and easy queries
Document and chunk text with metadata	MySQL	Needed to show sources next to every answer
Chunk embeddings	FAISS index file on disk	Fast similarity search
Uploaded PDF and text files	Private server folder, not served directly	Large files are better kept outside the database
Backups	Scheduled MySQL dump plus a copy of the FAISS index and uploads folder	The index can be rebuilt from MySQL and the files if it is lost

2.4.2 Schema

Every table uses foreign keys and constraints, so malformed records are rejected when written. Role, status and category accept only their allowed values, and timestamps are stored in UTC.

Table	Key fields	Indexes	Notes
users	user_id, name, email, password_hash, role, status	email (unique); role	Passwords hashed with bcrypt; role is user or admin
chat_sessions	session_id, user_id, title, created_at	user_id	One chat holds many questions
queries	query_id, session_id, question_text, asked_at	session_id + asked_at	Question exactly as submitted
answers	answer_id, query_id, answer_text, uncertain, model_name, latency_ms	query_id (unique)	One answer per question; kept for audit
answer_sources	answer_id, chunk_id, rank_no, score	answer_id; chunk_id	Links an answer to the chunks used
documents	doc_id, title, category, source_name, sha256, status, uploaded_by	sha256 (unique); category	Records provenance as described in Section 2.1.1

Table	Key fields	Indexes	Notes
chunks	chunk_id, doc_id, chunk_no, chunk_text, page_no, section_ref, faiss_pos	doc_id; faiss_pos (unique)	Rebuilt when a document is re-indexed
feedback	feedback_id, answer_id, rating, comment	answer_id	Helps review weak answers
audit_logs	log_id, actor_id, action, entity, timestamp	timestamp; actor_id	Logins, uploads, index rebuilds and user changes

2.5 UML and Behavioural Design

2.5.1 Use Case Diagram

The diagram shows who can do what. General users register, log in, ask questions, view answers with their sources, check chat history and give feedback. Administrators can do the same and also upload legal documents, rebuild the index, manage users and look at logs and feedback. Uploading a document always includes building the index, and giving feedback only makes sense once an answer is shown, which is why that use case extends “View answer and sources”.

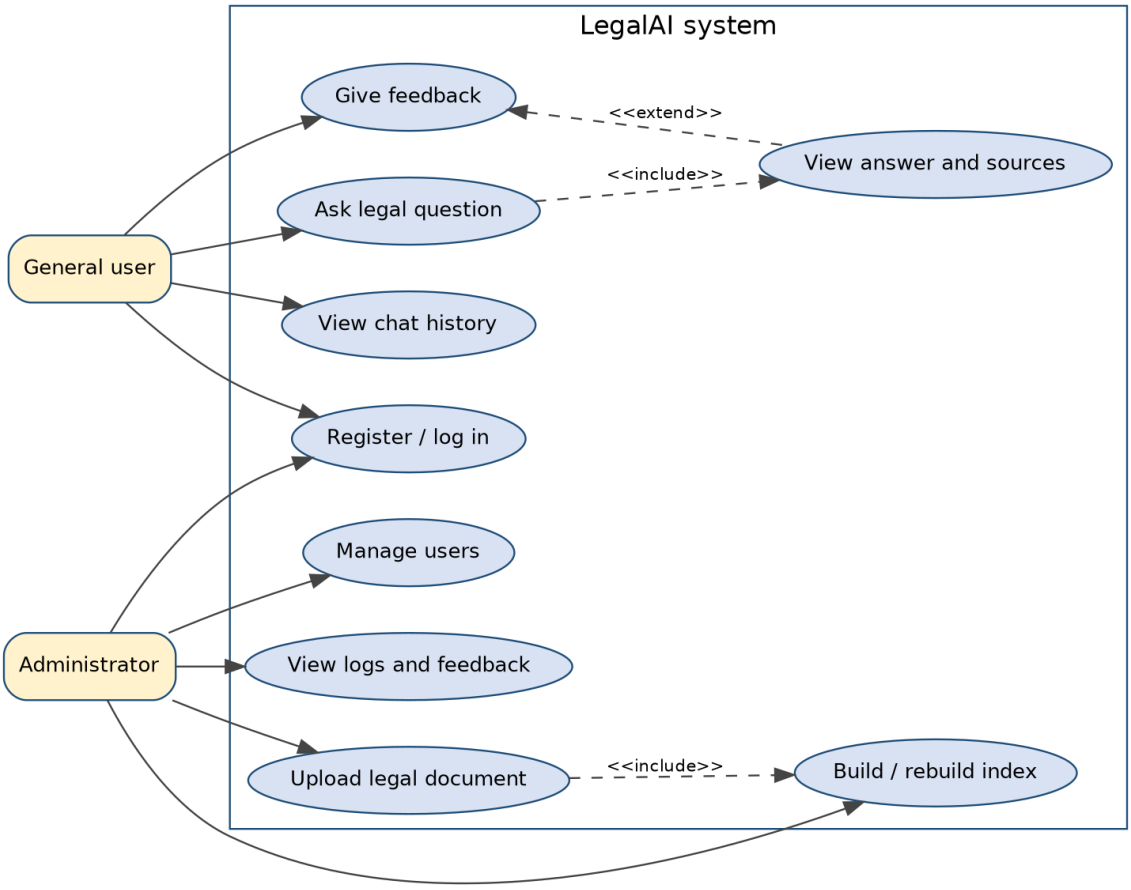


Figure 2.5.1: Use case diagram

2.5.2 Class / Component Diagram

The classes are grouped in layers. The controller exposes the REST endpoints and calls the services. RagService decides how a question is answered, Retriever embeds text and searches FAISS, GeminiClient calls the language model with timeouts and retries, and IngestionService prepares new documents. Repository wraps all MySQL access and Auth handles password hashing and roles. Each service is used through an interface, so the model, the vector index or the database can change without touching the controller.

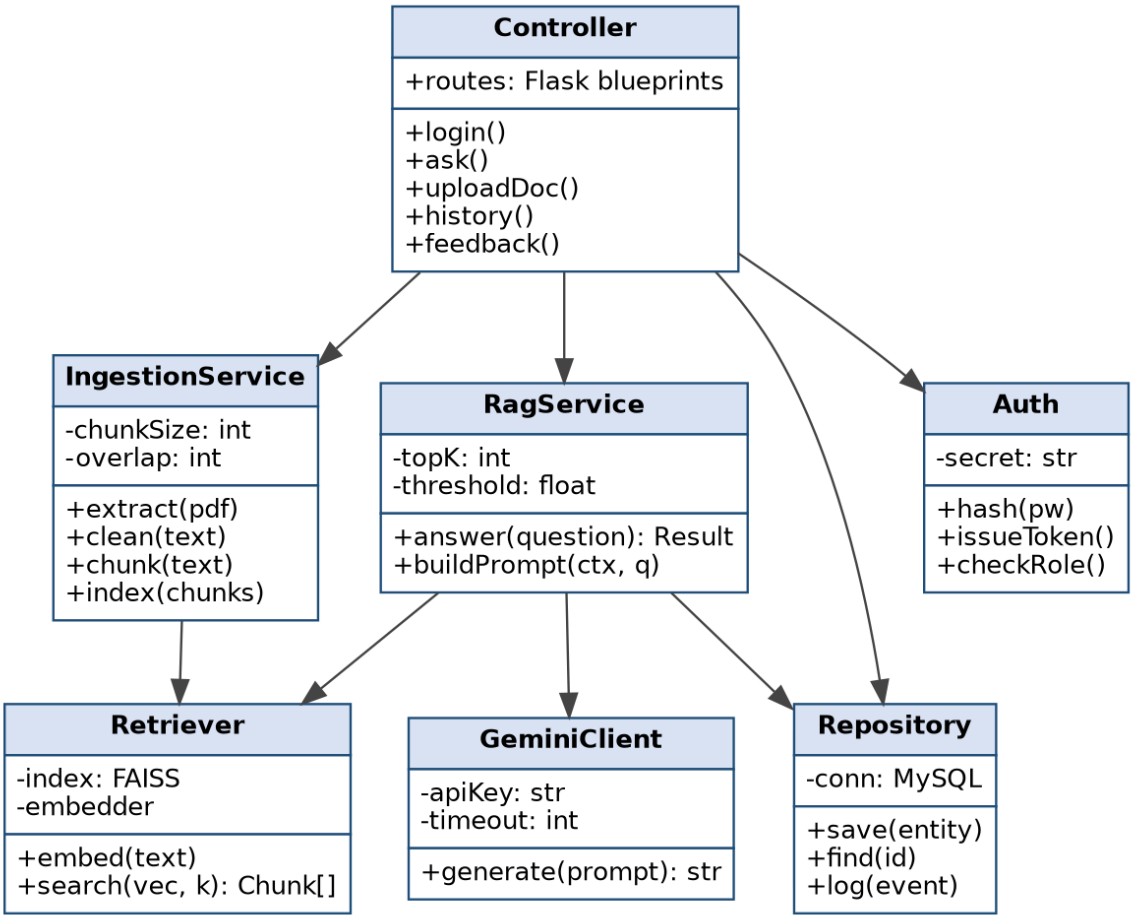


Figure 2.5.2: Class and component diagram

The REST endpoints that the front end uses are listed below. Every endpoint except register and login requires a valid JWT.

Method and endpoint	Allowed roles	Purpose
POST /api/auth/register	Anyone	Creates a user account
POST /api/auth/login	Registered user	Checks credentials and returns a JWT
POST /api/ask	User, admin	Takes a question, runs RAG and returns the answer with sources
GET /api/history	User, admin	Lists the caller’s chats and questions
POST /api/feedback	User, admin	Saves a rating for an answer
POST /api/admin/documents	Admin	Uploads a document with title, category and source
POST /api/admin/reindex	Admin	Rebuilds or updates the FAISS index

Method and endpoint	Allowed roles	Purpose
GET /api/admin/logs	Admin	Shows audit logs and feedback summary

2.5.3 Sequence Diagram

Here is how one question moves through the system. The API checks the token and the input, embeds the question and asks the retriever for the closest chunks. The chunk text and metadata are read from MySQL, a prompt is built and sent to Gemini, and the answer is saved with its sources before it is returned. The reader sees the answer and the source panel together and can rate the answer. If no chunk reaches the threshold, the call to Gemini is skipped and a fallback reply is returned.

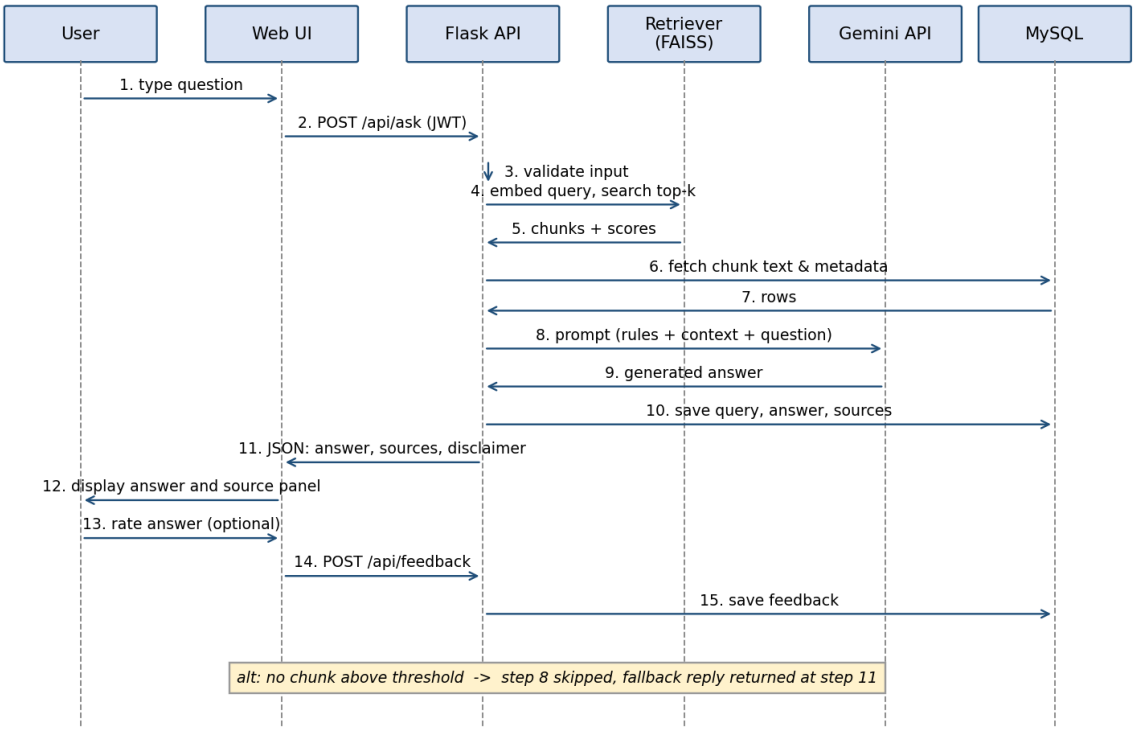


Figure 2.5.3: Sequence diagram

2.5.4 Activity Diagram

There are two paths. The indexing path, run by the administrator, covers upload, validation, text extraction, chunking, embedding and saving to FAISS and MySQL. A bad file is rejected and logged. The query path runs on every question: input validation, retrieval, the relevance check, the call to Gemini with timeouts and retries, and the display of the answer. If Gemini stays unavailable after the retries, the user sees a clear message and the question is still saved.

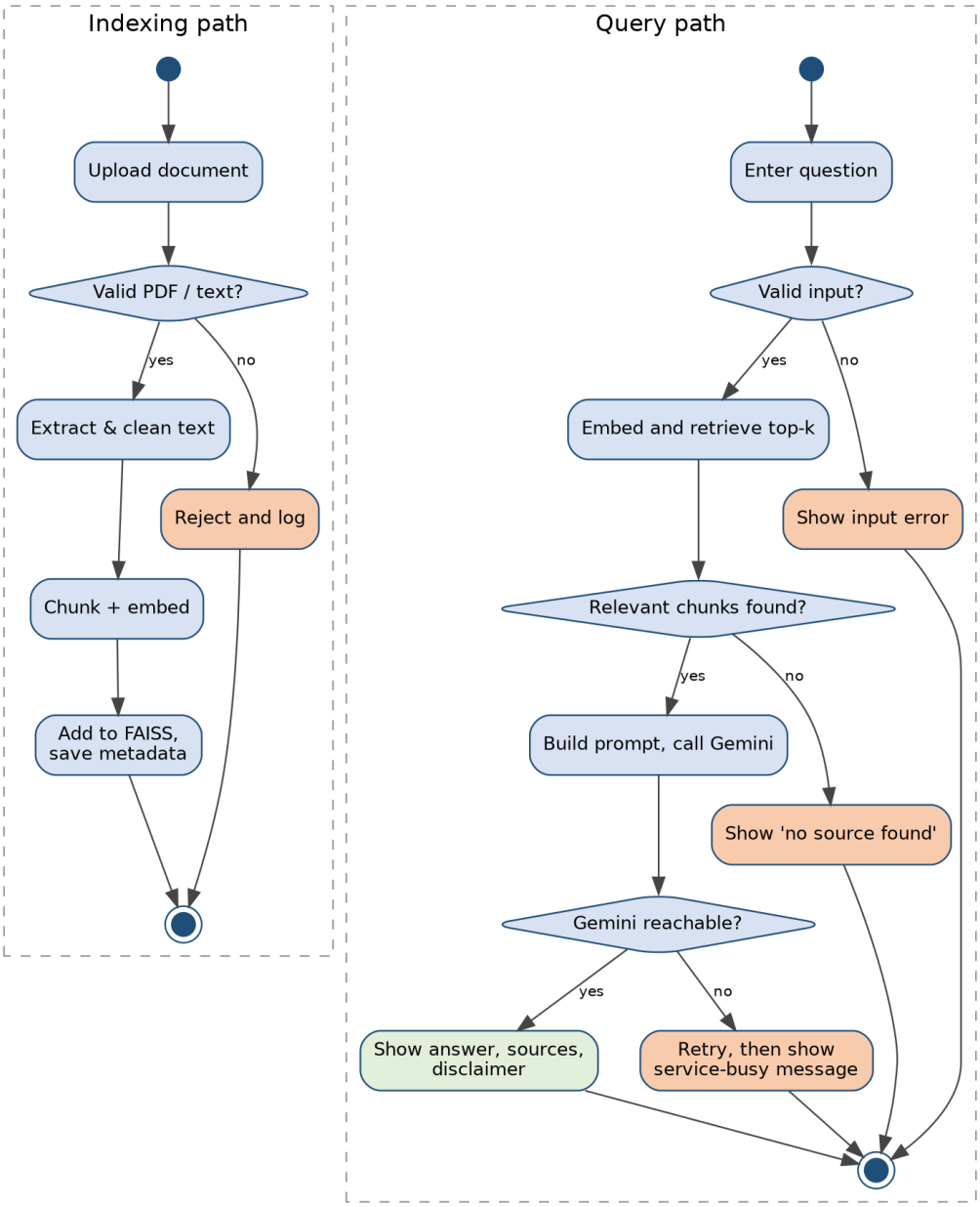


Figure 2.5.4: Activity diagram

Errors are returned in a consistent JSON format, and the front end shows the message in plain language:

Situation	HTTP status	Message shown to the user
Empty or too long question	400	Please enter a question of up to 1,000 characters.
Unsupported file type on upload	415	Please upload a valid PDF or text file.
File larger than the allowed size	413	This file is too large. Please upload a smaller file.
Not logged in or token expired	401	Please log in again to continue.
User calls an admin endpoint	403	You do not have permission to do this.
No relevant source above the threshold	200 (uncertain)	No relevant source was found in the knowledge base. Please rephrase or consult a legal professional.

Situation	HTTP status	Message shown to the user
Gemini API unavailable after retries	503	The assistant is busy right now. Your question is saved; please try again shortly.
Unexpected server error	500	Sorry, something went wrong. Please try again.

2.6 User Interface Design

The screens are designed for two groups. General users are often not technical, so the chat screen is a single page with a clear question box, the answer in the middle and the sources beside it. Chat history is in a side panel. Administrators need more detail, so their screen has an upload form, a document table with status and a rebuild-index button. AI output is always shown in italics with a note that it is general information and not legal advice. The wireframes below show the planned layout; they will be replaced with screenshots of the working app.

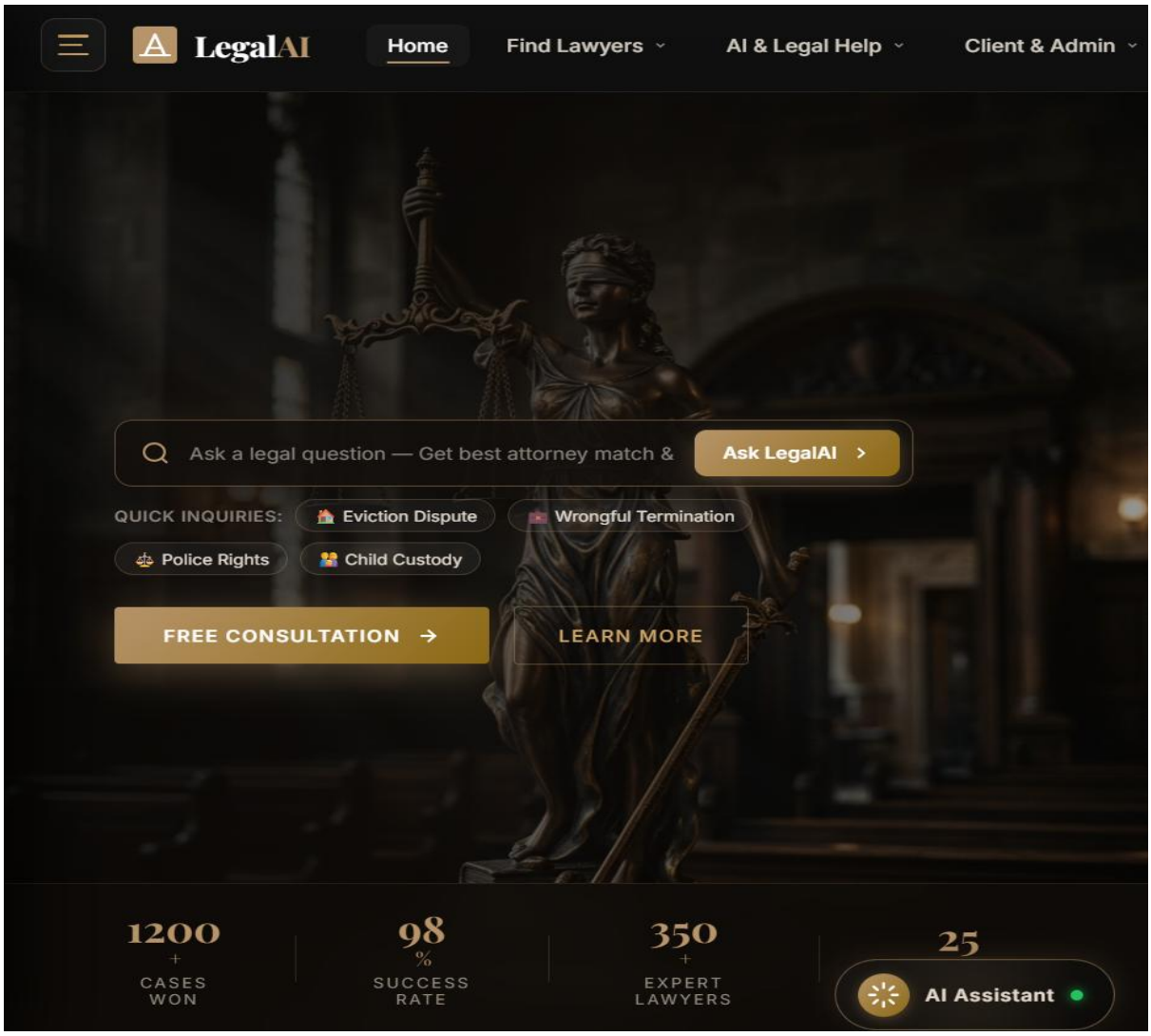


Figure 2.6.1: Home Page

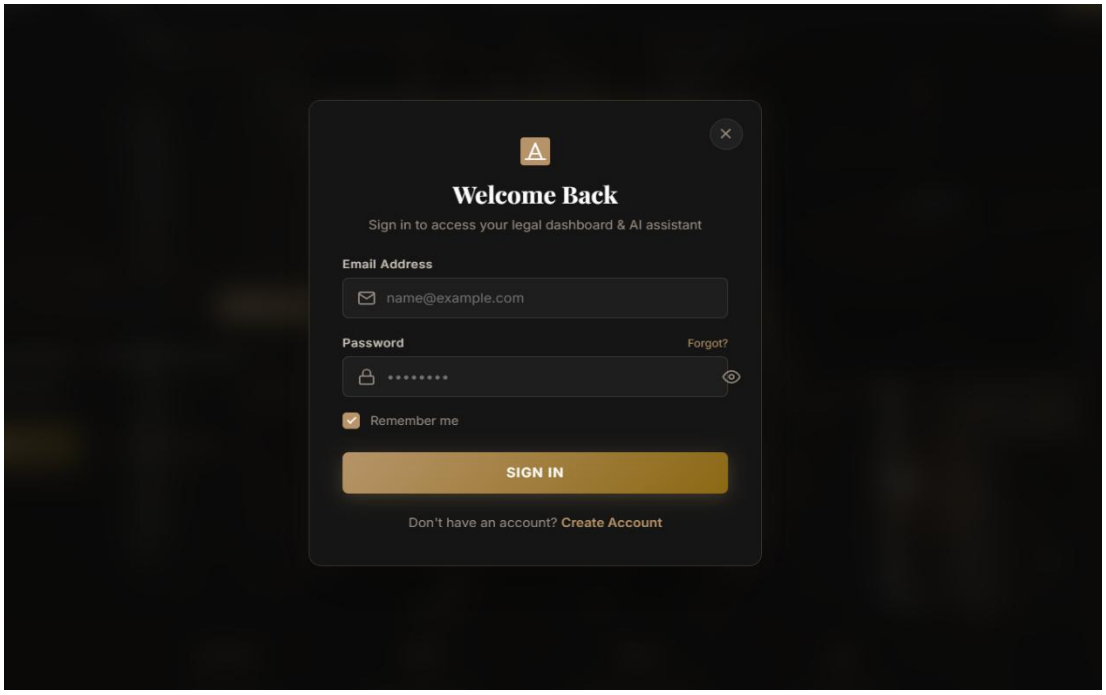


Figure 2.6.2: Login Page

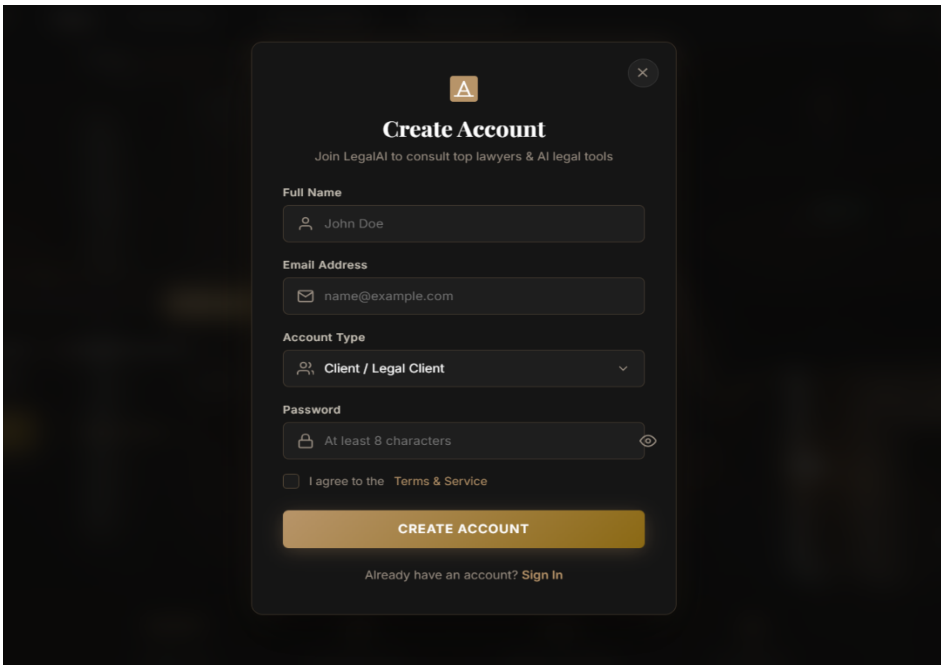


Figure 2.6.3: Register Page

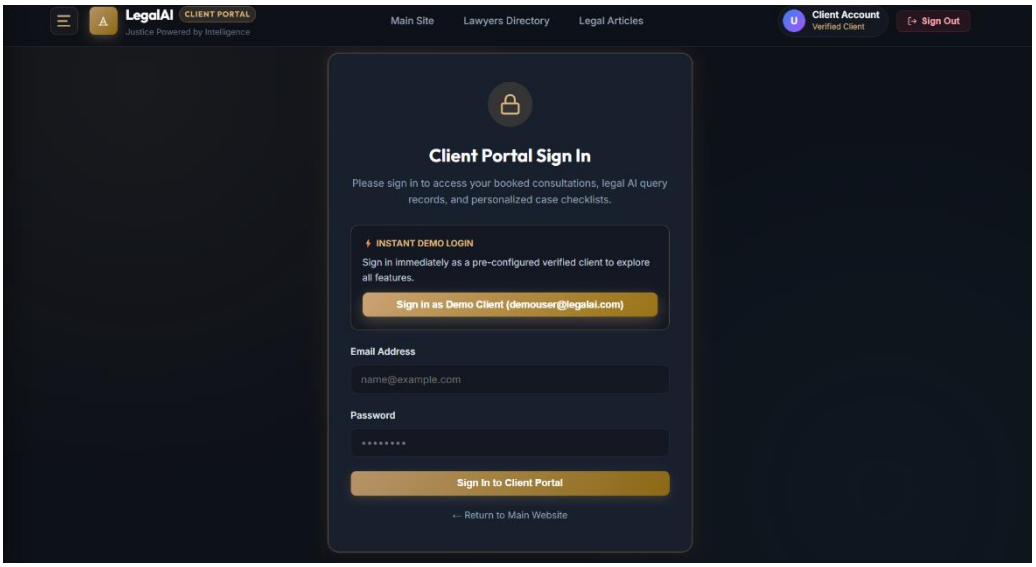


Figure 2.6.4: Clint Portal Page

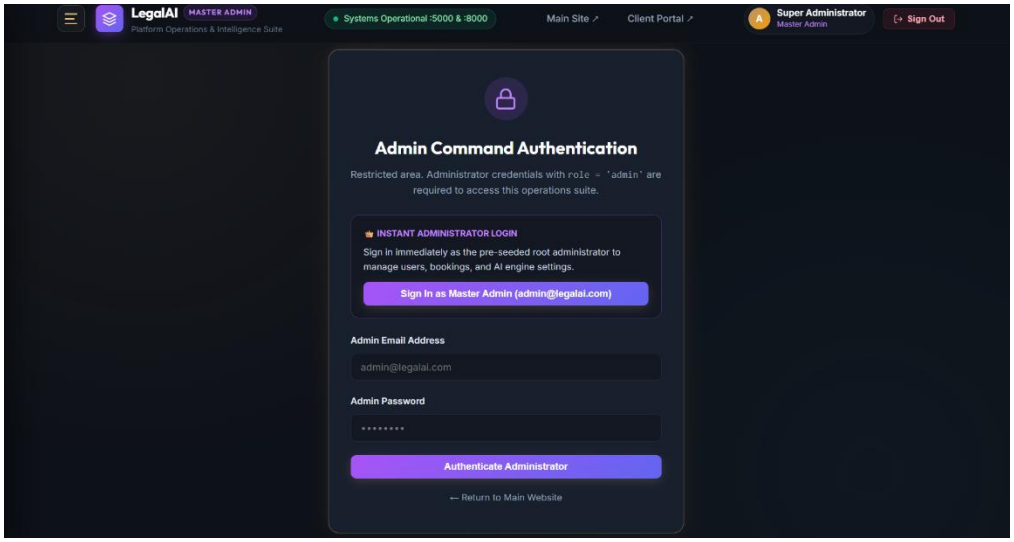


Figure 2.6.5: Admin Login Page

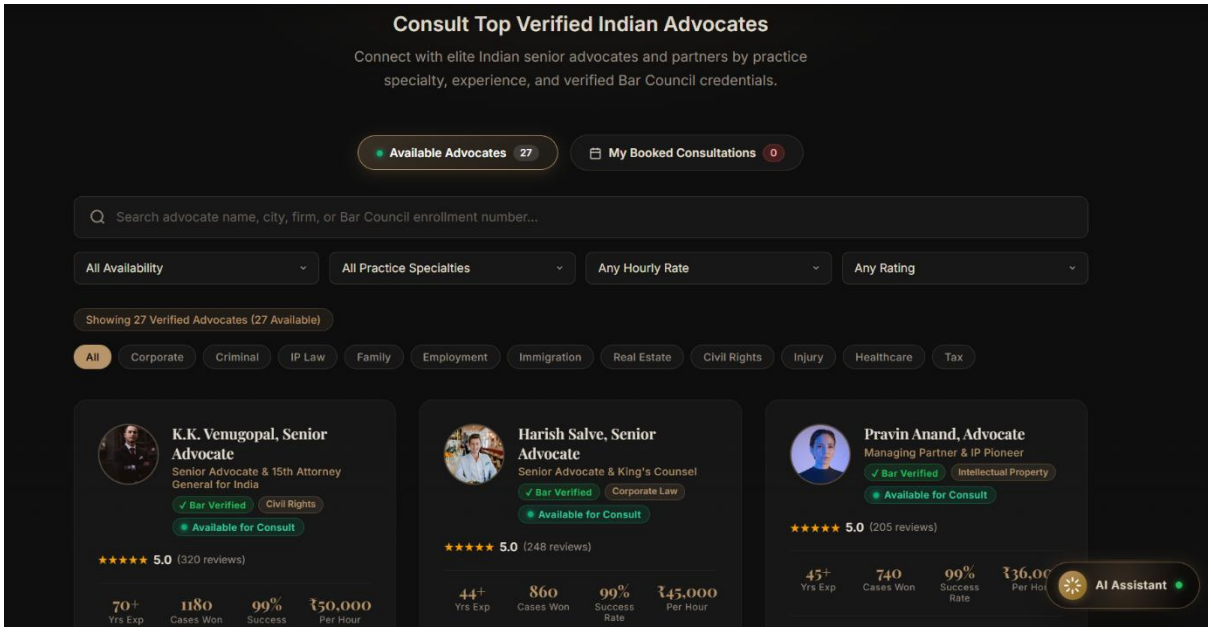


Figure 2.6.6: Advocates Page

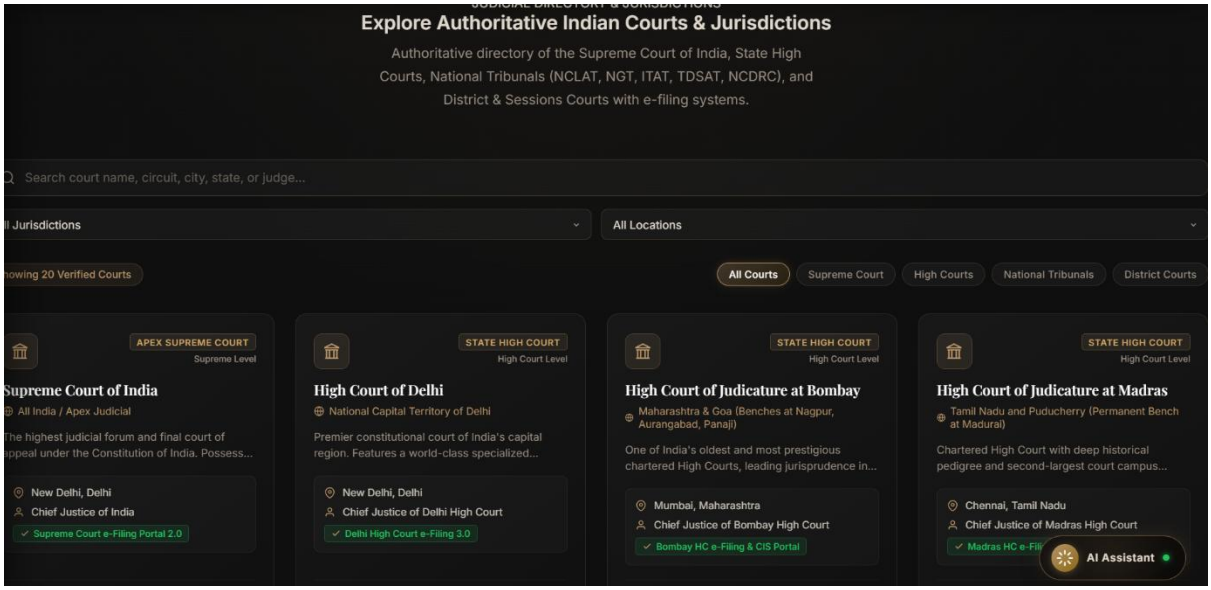


Figure 2.6.7: Court Page

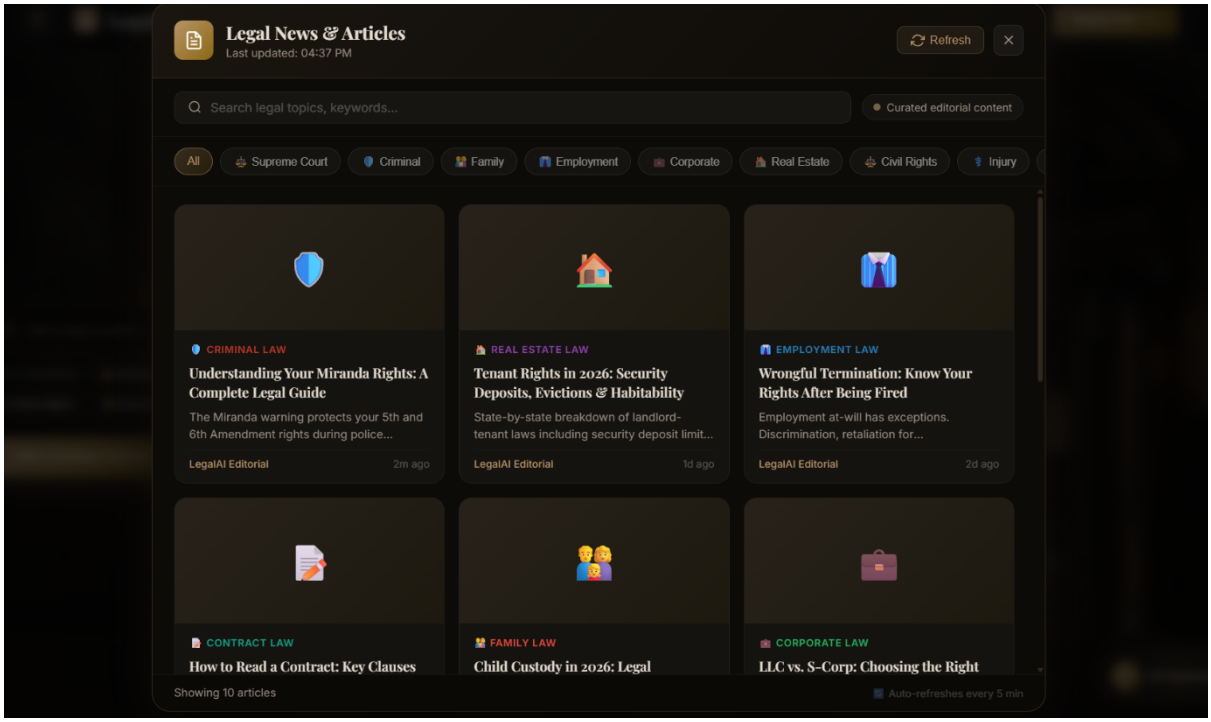


Figure 2.6.8: Legal News & Articles Page

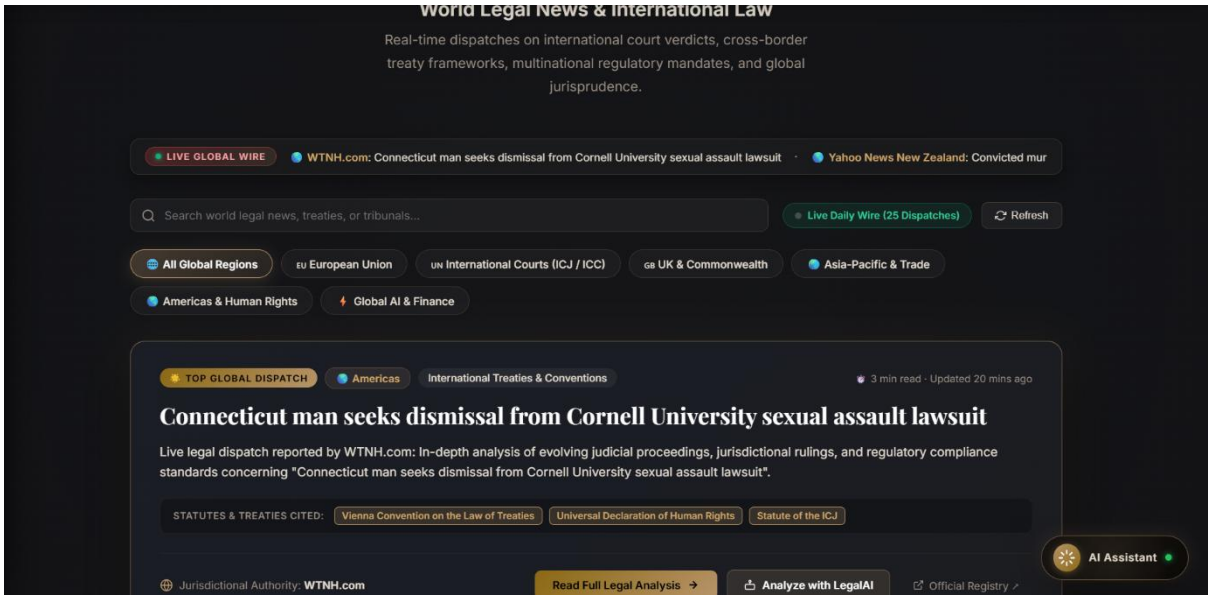


Figure 2.6.9: World News Page

3. References

[1] F. Redelaar, R. Van Drie, S. Verberne, and M. De Boer, "Attributed Question Answering for Preconditions in the Dutch Law," in Proc. Natural Legal Language Processing Workshop 2024, 2024, pp. 154–165, doi: 10.18653/v1/2024.nllp-1.12.

[2] R. Kalra, Z. Wu, A. Gulley, A. Hilliard, X. Guan, A. Koshiyama, and P. Treleaven, "HyPA-RAG: A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications," in Proc. 2025 Conf. Nations of the Americas Chapter of the Assoc. for Comput. Linguistics: Human Lang. Technol. (NAACL: HLT – Industry Track), 2025, pp. 1036–1054.

[3] S. K. Nigam, S. K. Mishra, N. Shallum, K. Ghosh, and A. Bhattacharya, "AILQA: Evaluating AI-Driven Legal Question Answering Systems for the Indian Legal System," arXiv preprint arXiv:2607.18825v1, Jul. 21, 2026.

[4] X. Chi, W. Wang, Z. Zhang, A. Li, Y. Huang, Y. Wu, K. Kuang, C. Sun, X. Liu, F. Wu, and M. Xiong, "LegalAI research in the LLM era: Data, modelling and evaluation," Artificial Intelligence Review, vol. 59, no. 4, Art. no. 118, 2026.

[5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982–3992.

[7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.